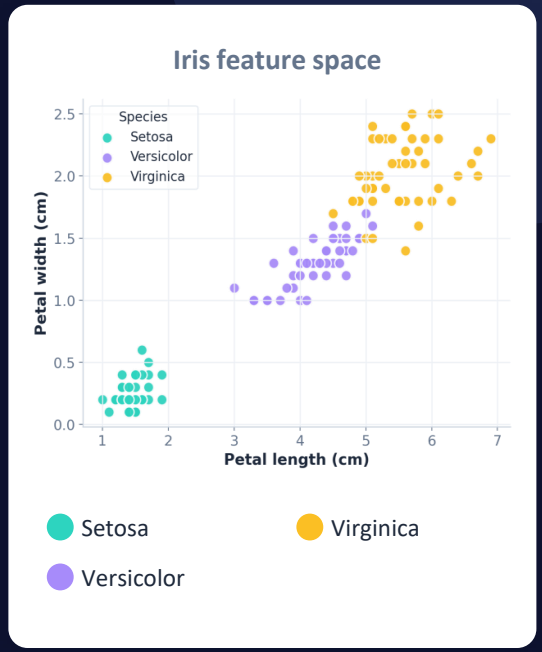
MACHINE LEARNING · CLASSIFICATION

Naive Bayes Classifier

Teaching a computer to recognise **Iris flowers** from their measurements — using probability.



BUILT WITH

scikit-learn · pandas · seaborn

THEORY · 1

What is Naive Bayes?

Naive Bayes is a simple, fast **machine-learning classifier** that makes predictions using **probability**. It asks: *"given these measurements, which class is most likely?"*

Educated guessing

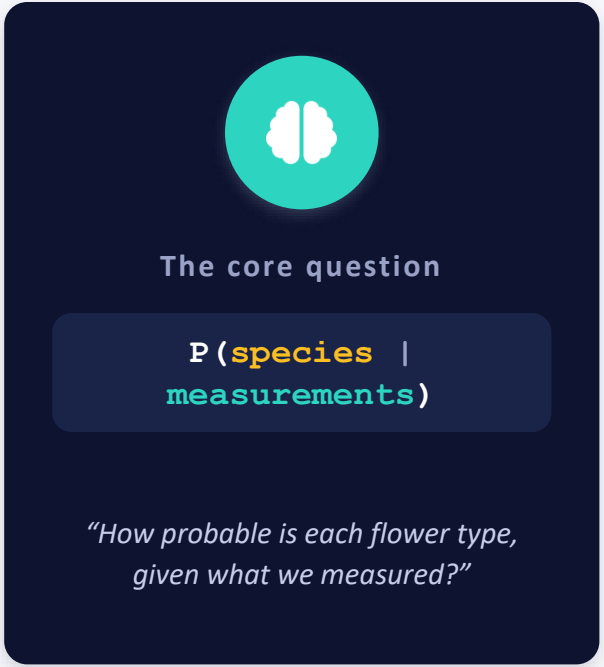
It scores every class by probability and picks the highest — a confident, fast guess.

Very few moving parts

No heavy tuning. It just learns simple statistics from the training data.

Surprisingly accurate

Despite its simplicity, it works remarkably well on clean problems like Iris.



THEORY · 2

Bayes' Theorem — the engine

$$P(y \mid X) = \frac{P(X \mid y) \cdot P(y)}{P(X)}$$

Posterior `P(y | X)`

what we want — prob. of a class

Likelihood `P(X | y)`

how well the data fits that class

Prior `P(y)`

how common that class is overall

Evidence `P(X)`

overall chance of seeing this data

In plain words

Start with how common each flower is **(prior).** Update that belief using how well the measurements match each flower **(likelihood).** The class with the highest result **wins.**

THEORY · 3

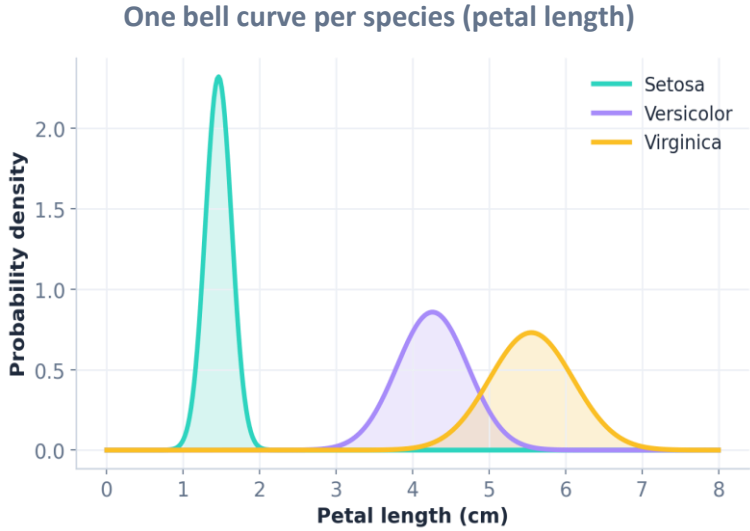
Why “Naive”, and why “Gaussian”?

“Naive” = features are independent

It assumes each measurement (petal length, width…) affects the answer on its own, ignoring how they relate. That’s rarely fully true — but it keeps the maths fast and still works well.

“Gaussian” = bell-curve features

Measurements are continuous numbers, so GaussianNB models each feature as a normal (bell) curve per class — learning a mean and spread, then asking which curve the new value fits best.



THEORY · 4

The Iris Dataset

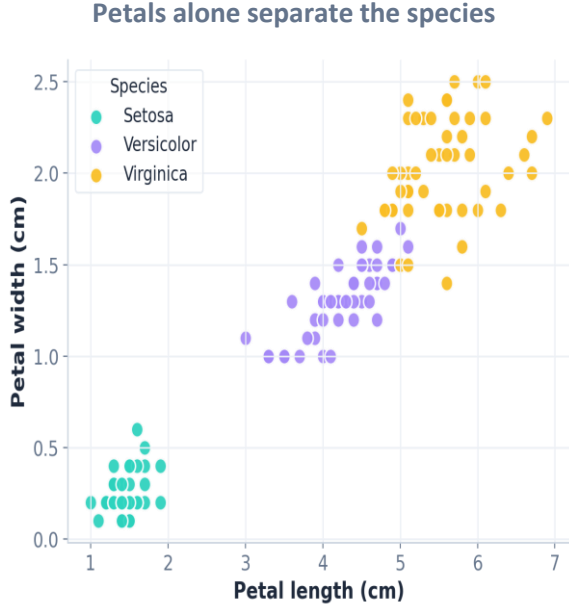
150	3	4
flowers	species	measurements

THE FOUR MEASUREMENTS (FEATURES → X)

- Sepal length
- Sepal width
- Petal length
- Petal width

THE ANSWER (TARGET → Y)

- 0 · Setosa
- 1 · Versicolor
- 2 · Virginica



THE WORKFLOW

8 steps from raw flowers to predictions

STEP 1

Import

STEP 2

Load data

STEP 3

Make table

STEP 4

Split

STEP 5

Train

STEP 6

Predict

STEP 7

Evaluate

STEP 8

Visualise

Next: every step explained line by line →

STEP 1 · CODE

Import the tools

imports.py

```
# the libraries we need
from sklearn import datasets
from sklearn.model_selection import (
    train_test_split)
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix)
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

LINE BY LINE

- **sklearn**
  ready-made datasets + the model & scoring tools.
- **train_test_split**
  splits data into learn vs. quiz sets.
- **GaussianNB**
  the Naive Bayes model itself.
- **pandas / seaborn / matplotlib**
  table handling + charts.

STEP 2 · CODE

Load the data

load.py

```
# load the famous Iris dataset
data = datasets.load_iris()

# X = inputs (measurements)
# Y = answers (the species)
X = data.data
Y = data.target
```

LINE BY LINE

- **`load_iris()`**
  loads the built-in flower dataset.

- **`X = data.data`**
  the inputs — the 4 measurements.

- **`Y = data.target`**
  the answers — species as 0 / 1 / 2.

STEP 3 · CODE

Build a readable table

table.py

```
# optional: view data as a table
df = pd.DataFrame(X,
    columns=data.feature_names)

# add a column for the species
df["Species"] = Y
```

LINE BY LINE

- **`pd.DataFrame(...)`**
  arranges X into a tidy, Excel-like table.
- **`columns=feature_names`**
  names each column after its measurement.
- **`df["Species"] = Y`**
  adds the answer column for easy viewing.

STEP 4 · CODE

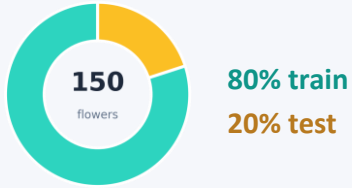
Split: train vs. test

split.py

```
# teach with 80%, quiz with 20%
X_train, X_test, Y_train, Y_test = (
    train_test_split(
        X, Y,
        test_size=0.2,
        random_state=42))
```

LINE BY LINE

- `test_size=0.2`
  keep 20% to quiz the model later.
- `random_state=42`
  fixes the split so runs are repeatable.
- `4 outputs`
  train inputs/answers + test inputs/answers.



STEP 5 · CODE

Create & train the model

train.py

```
# create an empty Naive Bayes model
model = GaussianNB()

# fit = train on the 80% set
model.fit(X_train, Y_train)
```

LINE BY LINE

- **`GaussianNB()`**
  creates a fresh, empty model.

- **`.fit(X_train, Y_train)`**
  “training” — it studies examples + answers and learns each species’ bell curves.

STEP 6 · CODE

Make predictions

predict.py

```
# guess species for the test flowers
predict = model.predict(X_test)

# show the guesses (0, 1 or 2)
print(predict)
```

LINE BY LINE

- `.predict(X_test)`
  feeds unseen flowers to the model.
- `predict`
  holds its guesses as 0 / 1 / 2.
- `print(predict)`
  shows the predicted species numbers.

STEP 7 · CODE

Check how well it did

evaluate.py

```
# what % did we get right?
print(accuracy_score(
    Y_test, predict) * 100)

# detailed per-class score card
print(classification_report(
    Y_test, predict,
    target_names=data.target_names))
```

LINE BY LINE

- **`accuracy_score`**
  % of guesses that were correct.
- **`classification_report`**
  per-species precision & recall.
- **`target_names`**
  prints flower names, not numbers.

100% accuracy on the 30 test flowers
(this run, random_state=42)

STEP 8 · CODE

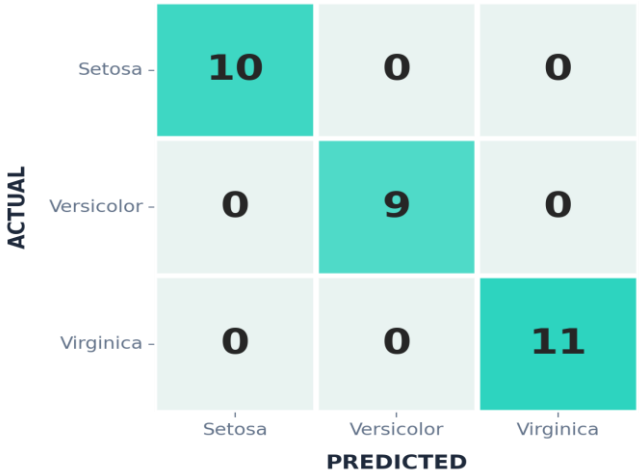
Draw the confusion matrix

matrix.py

```python
# grid of right vs. confused guesses
cm = confusion_matrix(Y_test, predict)

plt.figure(figsize=(8, 6))
sns.heatmap(cm,
    annot=True, fmt='d', cmap='Reds',
    xticklabels=data.target_names,
    yticklabels=data.target_names)
plt.xlabel("PREDICTED")
plt.ylabel("ACTUAL")
plt.show()
```

The output



Read it: the diagonal = correct guesses. All 30 land on the diagonal → **zero confusion.**

RECAP

What we built

Probability, not magic

Naive Bayes uses Bayes' theorem to pick the most likely species.

Learn → predict

Train on 80% of flowers, then quiz the model on the unseen 20%.

Measured the result

Accuracy + a confusion matrix confirmed every test flower was correct.

100%

test accuracy

GaussianNB on Iris